

STM32F103

ADC & DMA no STM32F103

Guia Tutorial Passo a Passo

Configuração, programação e boas práticas usando STM32CubeMX e HAL

Baseado em: RM0008 · STM32CubeF1 HAL · AN3116 · AN2834

O que você vai aprender neste tutorial

1

Características do ADC do STM32F103

Hardware, resolução, canais e modos disponíveis

2

Configuração no STM32CubeMX

Pinos, clock, parâmetros e geração de código

3

Leitura em Polling

Conversão sob demanda — simples, síncrona

4

Leitura em Modo Contínuo

Conversão livre, leitura assíncrona

5

Leitura com DMA ★ FOCO

Multi-canal, sem ocupar CPU — produção profissional

DESTAQUE

6

Conversão para Tensão e Filtros

Pós-processamento do valor digital

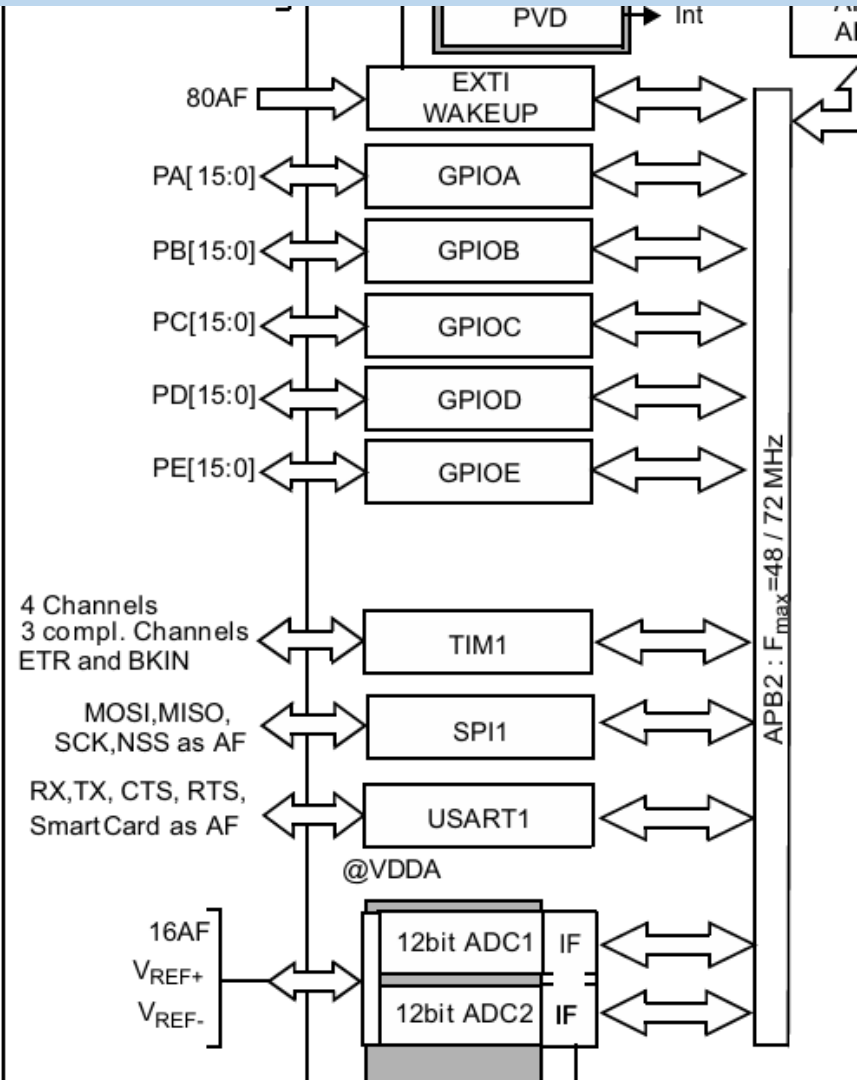
7

Boas Práticas de Hardware

Layout, desacoplamento e blindagem analógica

Características do ADC

Hardware interno do STM32F103 – o que temos disponível



Visão Geral do ADC – STM32F103



Resolução

12 bits → valores de
0 a 4095



Conversores

2 módulos independentes:
ADC1 e ADC2



Canais externos

Até 16 canais externos
(PA0–PA7, PB0–PB1, PC0–PC5)



Taxa máx.

1 MSPS
(mega-amstras por segundo)



Tensão ref.

0 a VDD-A
(tipicamente 3,3 V)



Modos

Único · Contínuo ·
Injetado · Scan

Modos de Conversão – Quando usar cada um?

Single / Polling

BÁSICO

 **Uso:** Leituras pontuais, debug, protótipo rápido

 **Vantagem:** Simples de implementar

 **Limitação:** Bloqueia a CPU durante a espera

Contínuo

INTERMEDIÁRIO

 **Uso:** Sensor sempre ativo, leitura frequente

 **Vantagem:** CPU lê quando quiser

 **Limitação:** Sem garantia de timing entre leituras

DMA (Scan Mode)

★ PRODUÇÃO

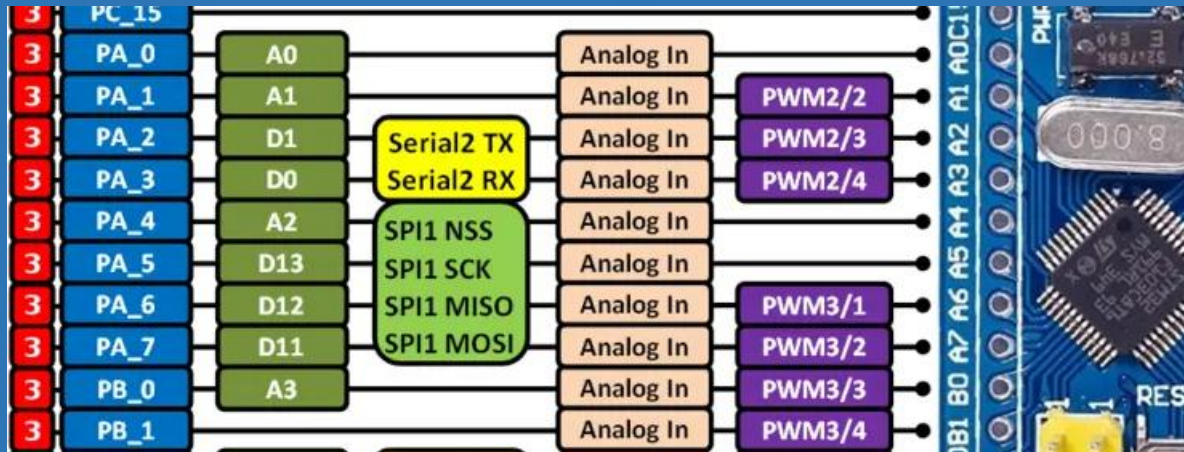
 **Uso:** Múltiplos canais, alta taxa, tempo real

 **Vantagem:** Zero intervenção da CPU após start

 **Limitação:** Configuração mais elaborada

Configuração no STM32CubeMX

Configurando pinos, clock e parâmetros do ADC



Passo 1 – Seleção de Pinos Analógicos

- 1 Abra o STM32CubeMX e crie um projeto para STM32F103C6T6A
- 2 Na aba Pinout & Configuration, clique no pino desejado (ex: PA0)
- 3 No menu pop-up selecione ADC1_IN0 (ou ADC2_INx para ADC2)
- 4 O pino ficará destacado — isso indica canal analógico configurado
- 5 Repita para cada canal adicional (ex: PA1 → ADC1_IN1, PA2 → ADC1_IN2)



Dica: Os pinos PA0–PA7, PB0–PB1 e PC0–PC5 são canais analógicos do STM32F103. Consulte o pinout do datasheet para confirmar o mapeamento.

Passo 2 – Configuração do Clock do ADC

REGRA CRÍTICA (RM0008 § 11.6)

O clock do ADC (ADCCLK) NÃO pode exceder 14 MHz.
Valor recomendado: 12 MHz (divisor 6 a partir de 72 MHz do SYSCLK).

- 1 Vá para Clock Configuration (aba superior)
- 2 Localize o bloco APB2 Peripheral Clocks — o ADC é derivado dele
- 3 Ajuste o divisor ADC Prescaler para /6 (resultado: 12 MHz para SYSCLK=72 MHz)
- 4 Verifique: o campo ADCCLK deve mostrar ≤ 14 MHz (verde)

Passo 3 – Parâmetros do ADC no CubeMX

PARÂMETRO	VALOR / INSTRUÇÃO
Mode	Independent mode (ADC1 e ADC2 independentes)
Data Alignment	Right (alinhamento à direita — bits 0 a 11)
Scan Conversion Mode	ENABLE — necessário para múltiplos canais com DMA
Continuous Conversion	ENABLE — mantém o ADC convertendo sem parar
Discontinuous Conv.	DISABLE (não usar junto com Contínuo)
Number of Conversion	Quantidade de canais que serão lidos (ex: 2)
Sampling Time	28.5 ou 41.5 ciclos — precisão × velocidade (ver slide seguinte)

Passo 4 – Tempo de Amostragem (Sampling Time)

O tempo de amostragem define por quantos ciclos de ADCCLK o capacitor interno do ADC carrega antes da conversão.

Tempo	Duração	Quando usar
1.5 ciclos	0.125 μ s @ 12MHz	Alta velocidade, sinal deve ter impedância muito baixa (<0,4 k Ω)
13.5 ciclos	1.1 μ s @ 12MHz	Equilíbrio geral para impedâncias médias
28.5 ciclos	2.4 μ s @ 12MHz	 Recomendado — bom para a maioria dos sensores
41.5 ciclos	3.5 μ s @ 12MHz	Fontes de alta impedância (ex: NTC, divisor resistivo)
71.5 ciclos	6.0 μ s @ 12MHz	Máxima precisão, fonte muito lenta (ex: célula de carga)

Leitura em Polling

Conversão simples, síncrona e bloqueante

Leitura em Polling – Código Completo

HAL_ADC_Start()



PollForConversion()



HAL_ADC_GetValue()



HAL_ADC_Stop()

```
uint32_t  adcValue;
float      volt;

/* 1. Inicia conversão ADC */
HAL_ADC_Start(&hadc1);

/* 2. Aguarda conversão (timeout = 10 ms) */
if (HAL_ADC_PollForConversion(&hadc1, 10) == HAL_OK) {

    /* 3. Lê o valor (0-4095) */
    adcValue = HAL_ADC_GetValue(&hadc1);

    /* 4. Converte para Volts: V = val * (3.3 / 4095) */
    volt = adcValue * (3.3f / 4095.0f);
}

/* 5. Para o ADC */
HAL_ADC_Stop(&hadc1);
```



Quando usar?

Prototipagem, debug, leitura única sob demanda



Cuidado!

A CPU fica bloqueada durante o Poll — não usar em laços críticos



Calibração

Antes do Start, chame:
HAL_ADCEx_Calibration_Start(&hadc1);

Leitura em Modo Contínuo

ADC converte sem parar — CPU lê quando quiser

Modo Contínuo – Configuração e Código

No modo contínuo, o ADC reinicia automaticamente após cada conversão. A CPU lê o último valor quando precisar — sem bloquear.

 **CubeMX: Continuous Conversion Mode → ENABLE | Scan Conversion Mode → DISABLE | DMA → DISABLE**

```
Inicialização (main)
/* No main(), ANTES do while */
HAL_ADCEx_Calibration_Start(&hadc1);
HAL_ADC_Start(&hadc1); /* inicia e deixa rodando */
```

```
Loop principal
/* Dentro do while(1) */
adcValue = HAL_ADC_GetValue(&hadc1);
volt = adcValue * (3.3f / 4095.0f);
HAL_Delay(10); /* opcional */
```



Vantagem

CPU nunca fica bloqueada esperando conversão



Atenção

Um único canal — para múltiplos use DMA com Scan Mode



Timing

O ADC converte continuamente; GetValue lê o último resultado disponível

Leitura com DMA

O método profissional: múltiplos canais, zero uso de CPU, tempo real



CPU Livre

O DMA transfere os dados do ADC para a RAM automaticamente. A CPU não precisa intervir.



Multi-canal

Com Scan Mode, todos os canais são lidos em sequência e armazenados em vetor.



Determinístico

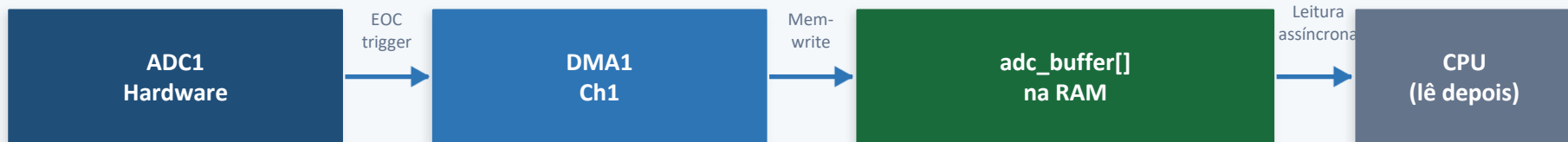
Cada amostra chega no buffer em tempo preciso — ideal para sistemas de controle.



Circular

Modo circular: o buffer se preenche, volta ao índice 0 e repete — sem parar.

DMA – Como Funciona Internamente?



- 1 O ADC1 converte um canal e levanta o flag EOC (End Of Conversion).
- 2 O DMA1 Canal 1 é disparado automaticamente pelo EOC — sem código da CPU.
- 3 O DMA copia o valor de ADC1->DR para adc_buffer[i] na RAM.
- 4 Com Scan Mode, passa para o próximo canal e repete os passos 1–3.
- 5 Quando todos os canais forem convertidos, o DMA levanta a flag TC (Transfer Complete).
- 6 O callback HAL_ADC_ConvCpltCallback() é chamado — a CPU processa o buffer.

DMA – Configuração no STM32CubeMX (Passo a Passo)

ADC1 > Parameter Settings

1

Scan Conversion Mode: ENABLE

Continuous Conversion Mode: ENABLE

Number of Conversions: (número de canais, ex: 2)

2

ADC1 > Rank 1

Channel: Channel 0 (PA0)

Sampling Time: 28.5 Cycles

3

ADC1 > Rank 2

Channel: Channel 1 (PA1)

Sampling Time: 28.5 Cycles

DMA – Configuração no STM32CubeMX (Passo a Passo)

4

ADC1 > DMA Settings > Add

DMA Request: ADC1

Channel: DMA1 Channel 1

Direction: Peripheral To Memory

Mode: Circular

Data Width Peripheral: Word (32-bit)

Data Width Memory: Word (32-bit)

5

NVIC Settings

DMA1 Channel1 global interrupt: ENABLE

(para receber o callback de conversão completa)

* APENAS EM CASO DE USO DA INTERRUPÇÃO

DMA – Código Completo (main.c)

Declarações e Inicialização

```
/* Variáveis globais */
uint32_t adc_buffer[2]; /* [0]=PA0, [1]=PA1 */

/* No main(), antes do while */
HAL_ADCEx_Calibration_Start(&hadc1);

HAL_ADC_Start_DMA(&hadc1, adc_buffer, 2);

/* while(1) – não precisa de nada!
   O DMA atualiza adc_buffer[] sozinho */
```

Callback de Conversão Completa

```
/* Chamado automaticamente pelo DMA
   quando o buffer está cheio */
void HAL_ADC_ConvCpltCallback(
    ADC_HandleTypeDef *hadc) {

    if (hadc->Instance == ADC1) {

        /* Lê canais */
        uint32_t ch0 = adc_buffer[0];
        uint32_t ch1 = adc_buffer[1];

        /* Converte para Volt */
        float v0 = ch0*(3.3f/4095.0f);
        float v1 = ch1*(3.3f/4095.0f);

    }
}
```

DMA – Modo Half-Transfer: Processamento em Tempo Real

Com buffers maiores (ex: 10 amostras por canal), é possível processar metade do buffer enquanto a outra metade ainda está sendo preenchida — ideal para filtros e médias.

Buffer 0..9 (Primeira Metade)
HALF-TRANSFER callback ← CPU processa aqui

Buffer 10..19 (Segunda Metade)
FULL-TRANSFER callback ← CPU processa aqui

```
/* Metade do buffer preenchida */
void HAL_ADC_ConvHalfCpltCallback(
    ADC_HandleTypeDef *hadc) {
    /* Processa adc_buffer[0..N/2-1] */
}

/* Buffer completo – segunda metade */
void HAL_ADC_ConvCpltCallback(
    ADC_HandleTypeDef *hadc) {
    /* Processa adc_buffer[N/2..N-1] */
}
```



Buffer size

Declare: `uint32_t adc_buffer[NUM_CH * N_SAMPLES];`
Ex: 2 canais × 10 amostras = 20 elementos



Média no callback

Calcule a média dentro do callback para filtragem simples.
Não faça operações pesadas — o callback é ISR!



volatile

Declare o buffer como volatile se acessado fora do callback:
`volatile uint32_t adc_buffer[20];`

DMA – Mapeamento de Canais no Buffer

Com Scan Mode e DMA Circular, cada posição do buffer corresponde a um canal específico, sempre na mesma ordem configurada no CubeMX.

Rank (CubeMX)	Canal ADC	Pino GPIO	Índice no Buffer	Acesso em C
Rank 1	IN0	PA0	adc_buffer[0]	adc_buffer[0]
Rank 2	IN1	PA1	adc_buffer[1]	adc_buffer[1]
Rank 3	IN2	PA2	adc_buffer[2]	adc_buffer[2]
Rank 4	IN4	PA4	adc_buffer[3]	adc_buffer[3]

Conversão e Filtros

Do valor digital à grandeza física — com qualidade

Conversão para Tensão e Filtros Digitais

Fórmula geral: $V_{\text{tensão}} = \text{adcValue} \times (V_{\text{REF}} / (2^n - 1))$ onde $n = 12$ bits, $V_{\text{REF}} = 3,3$ V

```
/* === 1. Conversão direta === */
float volt = adcValue * (3.3f / 4095.0f);

/* === 2. Média Móvel (N=8 amostras) === */
#define N_AVG 8
uint32_t hist[N_AVG] = {0};
uint8_t idx = 0;
uint32_t sum = 0;

void HAL_ADC_ConvCpltCallback(ADC_HandleTypeDef *h) {
    sum -= hist[idx];
    hist[idx] = adc_buffer[0];
    sum += hist[idx];
    idx = (idx + 1) % N_AVG;
    float volt_avg = (sum / N_AVG) * (3.3f/4095.0f);
}

/* === 3. Filtro IIR (passa-baixa)  $\alpha=0.1$  === */
filtered = filtered*(1.0f -  $\alpha$ ) + newSample* $\alpha$ ;
```



Conversão direta

Rápido, sem overhead. Use quando o sinal já é estável.



Média Móvel

Remove ruído branco. Custo: N variáveis e N somas. Ótimo para sensores lentos.



Filtro IIR

Suaviza com α pequeno (0.05–0.2). Baixo custo computacional — ideal em embedded.

Boas Práticas de Hardware

Layout, desacoplamento e separação analógico/digital

Boas Práticas – Hardware e Software

Hardware

- Coloque capacitor de 100 nF (cerâmico) + 10 μ F (eletrolítico) em VDDA e VREF+
- Separe planos de terra: AGND (analógico) e DGND (digital) — conecte em um único ponto (star ground)
- Mantenha trilhas de sinal analógico curtas e longe de clock, PWM e trilhas de alta corrente
- Use resistor série de 1 k Ω antes do pino analógico para limitar corrente e filtrar RF

Software / HAL

- Sempre chame HAL_ADCEx_Calibration_Start(&hadc1) antes do primeiro Start
- Com DMA, declare o buffer como volatile se acessado no loop principal
- Para múltiplas leituras, prefira DMA Circular — evita re-chamar Start_DMA
- Não faça operações pesadas dentro do callback (é uma ISR) — use flags

Resumo – Comparativo dos Modos de Leitura

Critério	Polling	Contínuo	DMA ★
Uso de CPU	Alto (bloqueante)	Médio (leitura ativa)	Mínimo
Múltiplos canais	Manual (repetir)	Não nativo	✓ Scan Mode
Timing preciso	✗ Depende do código	Parcial	✓ Hardware garante
Complexidade config.	Baixa	Baixa	Média
Uso em produção	Não recomendado	Casos simples	✓ Recomendado
Callback/IRQ	Não	Não	✓ Sim
Filtros em tempo real	Difícil	Médio	✓ Natural

Referências Bibliográficas

RM0008 Rev.21

STM32F101xx, STM32F102xx, STM32F103xx Reference Manual

STMicroelectronics, 2021 — Capítulos 11 (ADC) e 13 (DMA)

STM32F103C8 DS

STM32F103x8, STM32F103xB Datasheet

STMicroelectronics — Tabelas de pinout e características elétricas

AN3116

STM32's ADC Modes and Their Applications

STMicroelectronics Application Note — Modos de conversão e DMA

AN2834

How to get the best ADC accuracy in STM32 microcontrollers

STMicroelectronics Application Note — Hardware e software para precisão

UM1318

STM32CubeF1 HAL Driver Description

STMicroelectronics — Documentação das funções HAL do STM32F1

UM1718

STM32CubeMX User Manual

STMicroelectronics — Guia do gerador de código